

<u>שם מוסד:</u> , .

<u>סמל מוסד:</u> .

<u>שם הפרויקט:</u> AirHockey Robot.

<u>שם הסטודנט:</u> .

<u>ת. ז. סטודנט:</u> .

<u>שם המנחה:</u> .

<u>תאריך הגשה:</u> 08/07/2026.

תוכן עניינים:

1. טופס הצעה לפרויקט מעבדה במערכות אוטונומיות 4
תיאור הבעיה: 4
תיאור הפתרון: 5
מפרט טכני: 6
תרשים מלבנים: 7
2. תקציר 8
רקע לפרויקט 8
מטרת המערכת 8
תיאור הפרויקט 8
קהל היעד 9
3. תצהיר סטודנט 9
4. מושגים רלוונטיים לפרויקט 9
5. תיאור הנושא 10
6. תיאור הבעיה / הצורך במוצר בשוק 10
מטרות ויעדים 10
7. ניתוח חלופות מערכתי 11
חלופות מכניות 11
חלופות לבקר 11
חלופות למנועים 12
חלופות לזיהוי הדיסקית 12
חלופות תוכנה וספריות 12
8. תיאור החלופה הנבחרת 13
9. הסבר על הבקר 14
10. הסבר רכיבים — חיישנים ואופן פעולה 15
מצלמה / ESP32-CAM 15
חיישן מגע לשער 15
חיישן תאוצה / IMU 16
חיישן מרחק 16
מנועי DC צהוב 16
דרייבר ULN2003 16
מנוע סרוו 17
רצועת GT2 וגלגלות 17
מסך מגע 17
רמקול Piezo 18
תאורה 18
11. תיאור הארכיטקטורה 18
12. תיאור פרוטוקולי תקשורת 19
13. הסבר ספריות 20
14. תהליך הפיתוח 20
15. תיעוד הפתרון 21
a. שרטוט חשמלי: 21
b. חיבורים ואופן פעולה: 22

c. תיעוד קטעי קוד עיקריים כולל הסברים:.....24
d. תרשים זרימה: 41
e. תרשים פונקציונאלי: 41
f. התייחסות למערכת הפעלה: 42
g. התייחסות לאבטחת מידע:.....42
16. פיתוחים עתידיים.....42
17. סיכום ומסקנות.....42
18. ביבליוגרפיה.....43
מסכי אפליקציה.....43
מסכי מסך מגע.....46

1.טופס הצעה לפרויקט מעבדה במערכות אוטונומיות

שם הסטודנט: .

ת. ז. סטודנט: .

סמל מוסד: .

שם המנחה: .

שם הפרויקט: AirHockey Robot.

תקציר הפרויקט: מנגנון XY רובוטי למשחק הוקי אוויר מול שחקן אנושי.

בפרויקט זה יפותח אב־טיפוס של שולחן הוקי אוויר חכם, שבו אחד השחקנים הוא רובוט אוטונומי. השחקן האנושי משחק מצד אחד של השולחן, ובצד השני פועל רובוט הנשלט על ידי מנגנון XY ומגיב בזמן אמת למהלך המשחק.

במהלך המשחק מצלמה המותקנת מעל השולחן מזהה את מיקום הדיסקית ואת כיוון תנועתה, ומחשבת נקודת מפגש מתאימה עם הרובוט. הנתונים נשלחים אל בקר ESP, והוא מזיז את מנועי ה־XY כך שהרובוט יגיע למיקום הדרוש. לאחר המפגש עם הדיסקית, מנוע סרוו מבצע דחיפה מבוקרת במטרה להחזיר את הדיסקית לכיוון שער היריב.

בנוסף לראייה הממוחשבת, המערכת משלבת חיישן מרחק לבדיקת מיקום השחקן ביחס לשולחן, חיישן תאוצה המותקן על יד השחקן לזיהוי עוצמת ומהירות הדחיפה, וחיישני מגע בשערים לזיהוי גולים. במהלך המשחק מוצגים הניקוד והגדרות המשחק במסך מגע, ובמקביל מופעלים רמקול ומטריצת לדים להצגת חיוויים קוליים וחזותיים.

המערכת משלבת בקרה על מנועים, חיישנים, ראייה ממוחשבת, ממשק משתמש ואלגוריתמי קבלת החלטות, ומהווה דוגמה למערכת אוטונומית הפועלת בזמן אמת בסביבה דינמית מול שחקן אנושי.

תיאור הבעיה:

משחק הוקי אוויר הוא סביבה דינמית ומהירה, הכוללת תנועה רציפה של דיסקית על משטח חלק, שינויי כיוון פתאומיים ואינטראקציה ישירה בין שחקן אנושי לבין מערכת המשחק.

פיתוח מערכת רובוטית המסוגלת לשחק הוקי אוויר מול אדם מחייב פתרון של מספר בעיות מורכבות:

- זיהוי מיקום הדיסקית בזמן אמת באמצעות מצלמה.
- חישוב כיוון התנועה של הדיסקית וחיזוי נקודת המפגש עם הרובוט.
- שילוב נתונים מחיישנים נוספים, כגון חיישן מרחק וחיישן תאוצה.

- קבלת החלטות מהירה בהתאם למהלך המשחק.
- הזזת הרובוט לנקודת XY מתאימה באמצעות מנועים.
- ביצוע תגובה פיזית לדיסקית בעזרת מנוע סרוו.
- זיהוי גולים באמצעות חיישני מגע בשערים.
- הצגת מצב המשחק והניקוד לשחקן.

בניגוד למערכות רובוטיות הפועלות בסביבה קבועה וצפויה, משחק הוקי אוויר כולל תנועה מהירה, שינויים בלתי צפויים וצורך בתגובה מיידית. לכן הפרויקט מהווה אתגר בתחום הבקרה, החיישנים והמערכות האוטונומיות בזמן אמת.

תיאור הפתרון:

הפתרון המוצע הוא פיתוח מערכת רובוטית אוטונומית לשולחן הוקי אוויר, שבה הרובוט משחק מול שחקן אנושי ומגיב בזמן אמת לתנועת הדיסקית.

המערכת תפעל במספר שלבים עיקריים:

1. קליטת נתונים מהסביבה

המערכת תאסוף נתונים ממספר רכיבים:

- מצלמה המותקנת מעל השולחן ותזהה את מיקום הדיסקית ואת כיוון תנועתה.
- חיישן מרחק שיזהה את מיקום השחקן ביחס לשולחן.
- חיישן תאוצה המותקן על פרק ידו של השחקן, וישמש לזיהוי עוצמת ומהירות הדחיפה.
- חיישני מגע בשערים, שישמשו לזיהוי גולים.

2. עיבוד וניתוח נתונים

לאחר קליטת הנתונים, המערכת תחשב את מיקום הדיסקית, כיוון התנועה שלה ונקודת המפגש המשוערת עם הרובוט. בנוסף, ייעשה שימוש בנתוני החיישנים כדי לשפר את הבנת מצב המשחק ואת תגובת הרובוט.

3. קבלת החלטות

בהתאם למיקום הדיסקית ולכיוון תנועתה, המערכת תקבע האם יש צורך בתגובה הגנתית או התקפית. כאשר הדיסקית מתקרבת לאזור הרובוט, תחושב נקודת XY שאליה הרובוט צריך להגיע כדי לפגוש את הדיסקית.

4. ביצוע תנועה ותגובה

בקר ה־ESP יקבל את נקודת ה־XY שחושבה, ויפעיל את מנועי המנגנון הרובוטי כדי להזיז את הרובוט למיקום המתאים. לאחר שהרובוט מגיע לנקודת המפגש, מנוע סרוו יבצע דחיפה מבוקרת של הדיסקית לכיוון שטח היריב.

5. ניהול מהלך המשחק

במהלך המשחק המערכת תזהה גולים באמצעות חיישני המגע, תציג את הניקוד ואת מצב המשחק במסך המגע, ותפעיל חיווים קוליים וחזותיים באמצעות רמקול ומטריצת לדים.

המערכת פועלת בלולאה סגורה:

חישה ← עיבוד ← החלטה ← תנועה ← חישה

כך מתקבלת מערכת המסוגלת לעקוב אחר מצב המשחק, להגיב לתנועת הדיסקית ולבצע פעולות בזמן אמת מול שחקן אנושי.

כללי המשחק (לצורך מדיניות הרובוט)

- המשחק מתקיים בין שחקן אנושי לבין רובוט אוטונומי. כל צד מגן על שערו, והמטרה היא למנוע מהדיסקית להיכנס לשער שלו ולהחזיר אותה לכיוון שער היריב.
- נקודה מתקבלת כאשר הדיסקית נכנסת לאחד השערים.
- במהלך המשחק אין לגעת בדיסקית ביד, אלא רק באמצעות המחבט/המגן של המשחק.
- המשחק מסתיים באחד משני מצבים, לפי בחירת השחקן בתחילת המשחק:
  - כאשר אחד הצדדים מגיע ל־7 נקודות.
  - לאחר 5 דקות משחק.
- הרובוט פועל רק בחצי השולחן שלו ואינו עובר את קו האמצע.
- תנועת הרובוט נקבעת לפי מיקום הדיסקית והנתונים המתקבלים מהמצלמה ומהחיישנים, ובהתאם לכך הוא מזיז את המחבט לנקודת המפגש המתאימה.

מפרט טכני:

המערכת מבוססת על בקר **ESP32**, האחראי על קבלת הנתונים מהרכיבים השונים, עיבוד המידע, תקשורת בין חלקי המערכת ושליטה על פעולת הרובוט.

רכיבי חישה ותצוגה

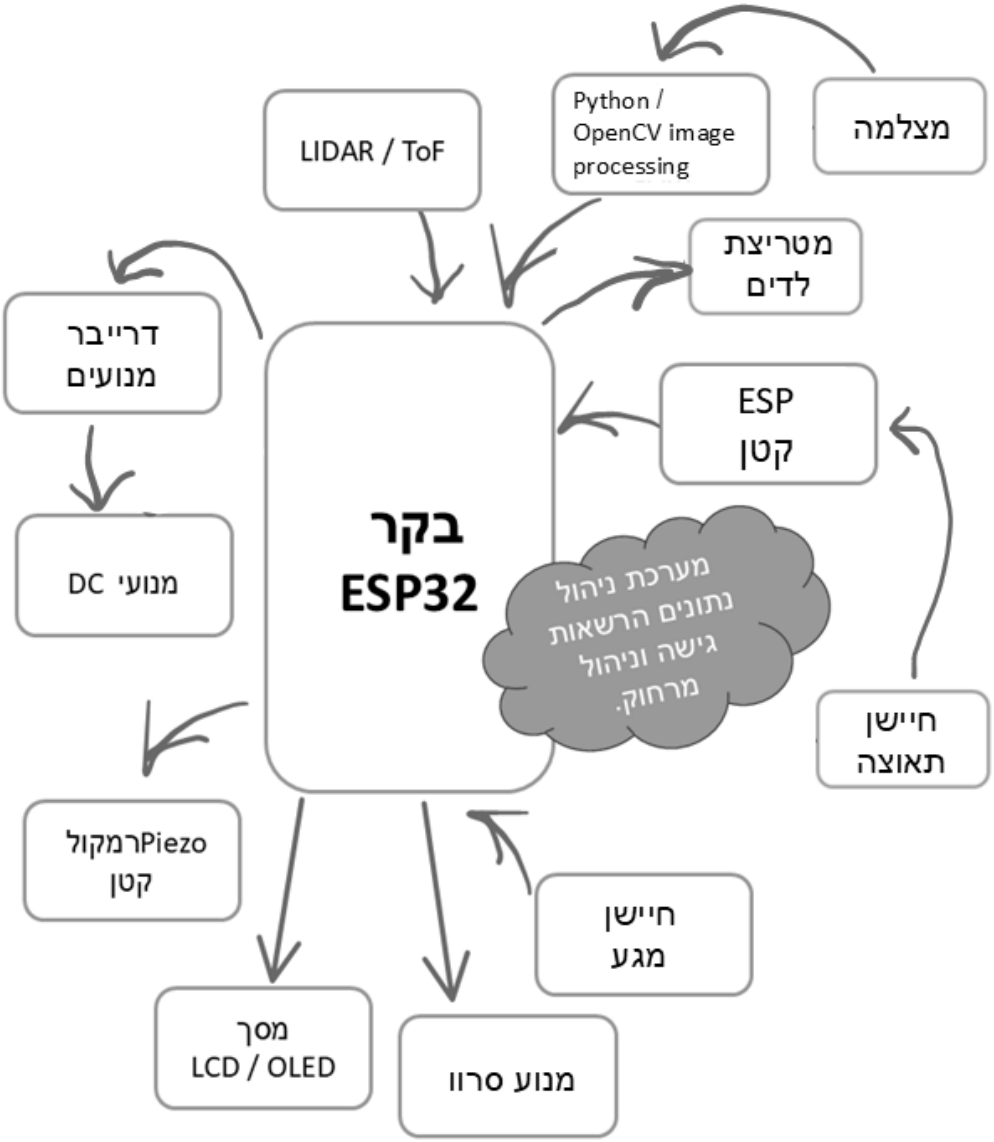
- **מצלמה / ESP32-CAM** – לזיהוי מיקום הדיסקית וכיוון התנועה שלה.
- **מסך מגע** - להצגת מצב המשחק, ניקוד השחקנים והגדרות המשחק.
- **חיישני מגע 24V–5** - לזיהוי כניסת הדיסקית לשער וזיהוי גול.
- **חיישן מרחק LIDAR / ToF** - למדידת מרחק השחקן מהשולחן.
- **חיישן תאוצה** - למדידת תנועת היד של השחקן ולהערכת עוצמת ומהירות דחיפת הדיסקית.
- **רמקול Piezo קטן** - להשמעת צלילים והתראות במהלך המשחק.
- **מטריצת לדים** - להצגת חיווים חזותיים בהתאם למהלך המשחק.

הנעת הרובוט

- הרובוט נע באמצעות **מנגנון XY**, המחזיק את המגן/מחבט המשחק ומזיז אותו בתוך חצי השולחן של הרובוט.
- לצורך תנועת המנגנון נעשה שימוש ב־**מנועי DC**, המאפשרים להזיז את הרובוט לנקודת ה־XY שחושבה על פי מיקום הדיסקית.
- לאחר שהרובוט מגיע לנקודת המפגש, **מנוע סרוו** מבצע דחיפה מבוקרת של הדיסקית לכיוון שטח היריב.

המנגנון נועד לאפשר לרובוט לחסום את הדיסקית, להגיב בזמן אמת למהלך המשחק ולהחזיר אותה לכיוון שער היריב.

תרשים מלבנים:



חתימת הסטודנט:

חתימת המנחה: '

תאריך: 03.03.26

2. תקציר

רקע לפרויקט

משחק הוקי אוויר הוא משחק מהיר ודינמי, הדורש בדרך כלל שני שחקנים פעילים. כאשר אין שחקן נוסף, לא ניתן ליהנות ממשחק מלא, והמשחק מאבד חלק גדול מהחוויה שלו. בנוסף, משחק מול אדם דורש תגובה מהירה, דיוק בתנועה וזיהוי מתמיד של מיקום הדיסקית, ולכן קשה לדמות אותו בצורה פשוטה ללא מערכת אוטונומית.

מעבר לכך, קיימת בעיה בהדגמה לימודית של מערכות רובוטיות. מערכות רבות בתחום הרובוטיקה והמערכות המשובצות מוצגות בצורה תאורטית או מופשטת, ולא תמיד מאפשרות לראות באופן ברור כיצד רובוט קולט נתונים מהסביבה, מקבל החלטה ומבצע פעולה פיזית בזמן אמת.

מטרת המערכת

מטרת המערכת היא לפתח שולחן הוקי אוויר חכם, שבו שחקן אנושי יכול לשחק מול יריב רובוטי באופן עצמאי. המערכת מאפשרת להזמין משחק מראש, לפתוח את המשחק באמצעות קוד אישי, לבחור רמת משחק וסוג משחק, ולאחר מכן לנהל משחק פעיל בצורה אוטומטית.

במהלך המשחק המערכת עוקבת אחר מצב הדיסקית והשחקן, מחשבת את תגובת הרובוט, ומזיזה את המגן שלו בהתאם למהלך המשחק. בנוסף, המערכת מזהה גולים, מעדכנת את הניקוד, מציגה את מצב המשחק לשחקן, ומוסיפה חיווי חזותי וקולי כדי ליצור חוויית משחק ברורה ומהנה.

תיאור הפרויקט

הפרויקט הוא שולחן הוקי אוויר חכם בגודל קטן של כ־51×31×10 ס"מ, שבו שחקן אנושי משחק מול רובוט אוטונומי. במקום שצריך שני שחקנים, המערכת מאפשרת לשחקן אחד ליהנות ממשחק מלא מול יריב רובוטי שמגיב בזמן אמת למהלך המשחק.

בצד אחד של השולחן נמצא השחקן האנושי, ובצד השני מותקן רובוט המחזיק את המגן של המשחק. הרובוט נע באמצעות מנגנון XY, כלומר מנגנון שמאפשר לו לזוז לשני כיוונים: ימינה ושמאלה, קדימה ואחורה. כך הוא יכול להגיע לנקודות שונות בחצי המגרש שלו, לחסום את הדיסקית ולהחזיר אותה לכיוון השחקן.

המנגנון פועל כך שהמערכת קובעת בכל רגע לאן הרובוט צריך להגיע במגרש, ולאחר מכן מפעילה את תנועת הרובוט בציר המתאים. הרובוט נע ימינה או שמאלה לפי הצורך, ובמקביל הוא נע קדימה ואחורה כדי להתמקם מול הדיסקית. לאחר שהרובוט מגיע למיקום המתאים, הוא משתמש במגן כדי לחסום את הדיסקית ומפעיל דחיפה לדיסקית להחזיר אותה לכיוון השחקן. בצורה זו נוצרת פעולה רציפה שבה הרובוט מגיב למהלך המשחק ומשמש כיריב פעיל מול השחקן האנושי.

במהלך המשחק המערכת מזהה את מצב הדיסקית בעזרת מצלמה וחיישנים, מחשבת לאן הרובוט צריך לזוז, ומפעילה את המנועים כדי להזיז את המגן למקום הנכון. כאשר יש גול, המערכת מעדכנת את הניקוד ומפעילה חיווי באמצעות מסך, לדים ורמקול.

המערכת מתוכננת כדגם לימודי וחווייתי המדגים בצורה ברורה כיצד רובוט יכול לקלוט נתונים מהסביבה, לקבל החלטה ולהגיב בתנועה. הדגש בפרויקט הוא על שילוב בין מכניקה, אלקטרוניקה, חיישנים, בקרה ותוכנה, ולא על בניית מוצר תחרותי במהירות גבוהה.

קהל היעד

קהל היעד של המערכת הוא משתמשים המעוניינים במשחק חווייתי ואינטראקטיבי מול רובוט, בעיקר ילדים, בני נוער וחובבי משחקים טכנולוגיים.

המערכת מתאימה גם למוסדות חינוך, מרכזי מדע ותערוכות טכנולוגיות, שבהם ניתן להציג בצורה מוחשית כיצד רובוט יכול לזהות מצב משחק, לקבל החלטות ולפעול בזמן אמת.

בנוסף, הפרויקט מתאים לתלמידים וסטודנטים הלומדים מערכות משובצות, רובוטיקה ובקרה, לצורך הדגמה מעשית של שילוב בין חיישנים, מנועים, עיבוד תמונה ותוכנה.

3. תצהיר סטודנט

אני החתום/ה מטה מצהיר/ה כי העבודה המוגשת במסגרת פרויקט זה בוצעה על ידי באופן עצמאי, ללא העתקה וללא שימוש בעבודה של אדם אחר שלא צוין במפורש.

כל החומרים, הרכיבים, הקוד, ההסברים והתרשימים המופיעים בספר הפרויקט נכתבו, נבנו או עובדו על ידי במסגרת עבודתי על הפרויקט. במקומות שבהם נעשה שימוש במקורות חיצוניים, ברכיבים מוכנים, ספריות תוכנה או דוגמאות קיימות, הדבר נעשה לצורך למידה, פיתוח ושילוב במערכת, תוך התאמה לפרויקט הנוכחי.

שם הסטודנט:
תעודת זהות:
חתימה: .
תאריך: 29/04/26

4. מושגים רלוונטיים לפרויקט

שולחן הוקי אוויר
שולחן משחק שבו דיסקית נעה על משטח חלק. בפרויקט נעשה שימוש בשולחן קטן לצורך בניית דגם רובוטי של משחק הוקי אוויר מול שחקן אנושי.

דיסקית
העצם הנע במשחק. הדיסקית נעה על פני משטח השולחן, והרובוט צריך להגיב למיקומה ולכיוון תנועתה.

מגן / Paddle
החלק שבאמצעותו פוגעים בדיסקית. בפרויקט אחד המגינים מוחזק ומוזז על ידי מנגנון רובוטי.

רמת משחק
רמת קושי המשחק משפיעה על אופן פעולת הרובוט, לדוגמה על מהירות התגובה, מהירות תנועת המגן

ורמת הדיוק בניסיון לחסום או להחזיר את הדיסקית. ככל שרמת המשחק גבוהה יותר, הרובוט פועל בצורה מהירה ומאתגרת יותר מול השחקן.

סוג משחק
המשתמש בוחר האם המשחק יתנהל לפי מספר ניצחונות/נקודות, או לפי זמן מוגדר מראש. בחירה לפי מספר ניצחונות מסיימת את המשחק כאשר אחד הצדדים מגיע לכמות הניצחונות שנקבעה. בחירה לפי מספר דקות מסיימת את המשחק כאשר הזמן שהוגדר מסתיים, והמנצח נקבע לפי הניקוד באותו רגע.

5. תיאור הנושא

נושא הפרויקט הוא פיתוח מערכת רובוטית אוטונומית למשחק הוקי אוויר מול שחקן אנושי. הפרויקט שייך לתחום המערכות המשובצות והרובוטיקה, ומשלב בין תוכנה, חומרה, חיישנים ובקרה.

המערכת צריכה לבצע כמה פעולות עיקריות:
1. לקלוט מידע מהסביבה באמצעות מצלמה וחיישנים.
2. לזהות את מיקום הדיסקית ואת כיוון תנועתה.
3. לחשב לאן יש להזיז את המגן הרובוטי.
4. להפעיל מנועי DC ולהזיז את מנגנון ה־XY.
5. להפעיל מנוע סרוו לצורך דחיפת הדיסקית.
6. לזהות גולים ולעדכן ניקוד ומצב משחק.
7. להציג את מהלך המשחק והגדרותיו במסך המגע.
8. לאפשר התחלת משחק לפי קוד משחק שהוזמן דרך האתר.

המערכת פועלת בלולאה חוזרת:

חישה ← **עיבוד** ← **החלטה** ← **תנועה** ← **בדיקה מחדש**.

6. תיאור הבעיה / הצורך במוצר בשוק

משחקים אינטראקטיביים מול אדם דורשים בדרך כלל שני שחקנים. כאשר אין שחקן נוסף, המשתמש אינו יכול ליהנות ממשחק מלא. בנוסף, מערכות לימוד רובוטיות רבות הן מופשטות, ולא תמיד מדגימות בצורה ברורה כיצד רובוט קולט נתונים מהסביבה ומגיב אליהם בזמן אמת.

הצורך בפרויקט הוא יצירת דגם מוחשי של משחק מול רובוט, המדגים בצורה פשוטה וברורה שילוב בין זיהוי עצם נע, תגובה בזמן אמת, בקרה על מנועים ותנועה דו־צירית.

הפרויקט נותן מענה גם לצורך לימודי, מכיוון שהוא מציג מערכת המשלבת מכניקה, אלקטרוניקה, חיישנים, תוכנה ובקרה בתוך משחק מוכר וחווייתי.

מטרות ויעדים

מטרות הפרויקט הן:

- בניית מערכת רובוטית קטנה וזולה יחסית.
- יצירת תנועה מכנית בשני צירים באמצעות מנגנון XY.
- בניית מנגנון המאפשר למגן להגיע לנקודות שונות בחצי המגרש של הרובוט.
- שילוב מצלמה לזיהוי מיקום הדיסקית וכיוון תנועתה.
- שימוש בחיישנים לזיהוי תאוצה, מרחק וגולים.
- שליטה במנועי DC ובמנוע סרוו באמצעות בקר ESP32.
- הצגת ניקוד, זמן ומצב משחק במסך המגע.
- יצירת בסיס למערכת משחק אוטונומית מול שחקן אנושי.

המערכת אינה מיועדת בשלב הראשון להיות מוצר מסחרי מהיר ומושלם, אלא דגם לימודי המדגים את עקרון הפעולה של מערכת רובוטית אוטונומית בסביבה דינמית.

7. ניתוח חלופות מערכתי

בשלב תכנון המערכת נבדקו מספר חלופות מבחינת רכיבי חומרה, ספריות תוכנה ואופן מימוש הקוד. מטרת הבדיקה הייתה לבחור פתרון המתאים לדגם לימודי, פשוט יחסית לבנייה, ועם יכולת תגובה בזמן אמת.

חלופות מכניות

זרוע רובוטית רגילה

בחלופה זו משתמשים בזרוע רובוטית עם מפרקים המחזיקה את המגן.
יתרון: נראית כמו יד רובוטית ומדמה תנועה אנושית.
חיסרון: איטית יותר, קשה לכיוון, פחות מדויקת לתנועה על משטח ישר, ועלולה לא להגיע לכל אזור חצי השולחן.

מנגנון XY עם מסילות ורצועות

בחלופה זו המגן נע על שני צירים באמצעות מוטות ליניאריים, מיסבים, רצועות וגלגלות.
יתרון: מתאים למשטח מלבני, פשוט להבנה, קל יחסית לשליטה ומאפשר תנועה מדויקת יותר.
חיסרון: דורש בנייה מכנית ישרה ומדויקת.

מנגנון מגנטי מתחת לשולחן

בחלופה זו מנוע נע מתחת לשולחן ומזיז מגנט, והמגן שמעל השולחן נמשך אחריו.
יתרון: משטח המשחק נשאר נקי ממנגנונים חיצוניים.
חיסרון: דורש התאמת מגנטים, תלוי בעובי השולחן ועלול לגרום להחלקה או איבוד אחיזה.

החלופה שנבחרה: מנגנון XY עם מסילות ורצועות, מכיוון שהוא מתאים למבנה השולחן, פשוט יחסית למימוש ומאפשר שליטה ברורה במיקום המגן.

חלופות לבקר

Arduino UNO

יתרון: פשוט, זול ונפוץ מאוד.
חיסרון: חלש יותר, ללא תקשורת WiFi מובנית, ופחות מתאים למערכת הכוללת תקשורת, חיישנים רבים וניהול משחק.

ESP32
יתרון: כולל WiFi, חזק יותר מארדואינו רגיל, מתאים לפרויקטים משובצים, מאפשר תקשורת עם שרת ומסוגל לשלוט בכמה רכיבים במקביל.
חיסרון: דורש מעט יותר ידע בתכנות ובהגדרות.

Raspberry Pi
יתרון: חזק יותר ומתאים לעיבוד תמונה מתקדם.
חיסרון: יקר יותר, צורך יותר חשמל, ודורש מערכת הפעלה, ולכן פחות מתאים לדגם פשוט הנשלט בזמן אמת.

החלופה שנבחרה: ESP32, מכיוון שהוא מספק איזון טוב בין מחיר, יכולות תקשורת, שליטה ברכיבים והתאמה למערכות משובצות.

חלופות למנועים

מנוע סרוו MG90S
יתרון: קל לשליטה בזווית ומתאים לפעולות קצרות ומדויקות.
חיסרון: אינו מתאים להנעה רציפה של מנגנון XY לאורך מסילה.

מנוע DC
יתרון: מהיר, פשוט יחסית להפעלה ומתאים להזזת מנגנון מכני באמצעות רצועות.
חיסרון: לצורך שליטה מדויקת במיקום נדרש מנגנון משוב, כגון אנקודר או חיישני גבול.

מנוע צעד 28BYJ-48
יתרון: זול, פשוט, ומאפשר תנועה בצעדים מוגדרים.
חיסרון: איטי ולא חזק במיוחד, ולכן פחות מתאים לתנועה מהירה של רובוט במשחק הוקי אוויר.

החלופה שנבחרה: מנועי DC להנעת מנגנון ה־XY, ובנוסף מנוע סרוו לדחיפה מבוקרת של הדיסקית.

חלופות לזיהוי הדיסקית

חיישני מרחק
יתרון: פשוטים וזולים.
חיסרון: אינם מאפשרים זיהוי מדויק ורציף של מיקום הדיסקית על כל שטח השולחן.

חיישני מגע
יתרון: מתאימים לזיהוי פגיעה או כניסת דיסקית לשער.
חיסרון: אינם מזהים את מיקום הדיסקית במהלך התנועה.

מצלמה
יתרון: מאפשרת זיהוי של מיקום הדיסקית וכיוון התנועה שלה על גבי המשטח.
חיסרון: דורשת עיבוד תמונה ותנאי תאורה מתאימים.

החלופה שנבחרה: מצלמה לזיהוי הדיסקית, בשילוב חיישני מגע לזיהוי גולים וחיישנים נוספים לשיפור הבנת מצב המשחק.

חלופות תוכנה וספריות

קוד עצמאי בסיסי

יתרון: מתאים לדרישות לימודיות, מאפשר שליטה מלאה על האלגוריתם, ומקל על הבנת אופן פעולת המערכת.
חיסרון: דורש כתיבה ידנית של חלקים רבים בקוד.

OpenCV

יתרון: ספרייה מתאימה לעיבוד תמונה, זיהוי צבעים וזיהוי עצמים.
חיסרון: דורשת משאבי עיבוד, ולעיתים עדיף להריץ אותה במחשב או בשרת ולא ישירות על ESP32 רגיל.

Firebase / שרת נתונים

יתרון: מאפשר שמירת נתונים, ניהול משחקים, קוד משחק, ניקוד והיסטוריה.
חיסרון: דורש תקשורת אינטרנט ופתרון בעיות חיבור.

החלופות שנבחרו: שימוש בקוד עצמאי לשליטה בבקר וברכיבים, שימוש בעיבוד תמונה לזיהוי הדיסקית, ושמירת נתוני משחק במערכת/שרת לצורך ניהול ומעקב.

8. תיאור החלופה הנבחרת

החלופה שנבחרה היא מערכת המבוססת על מנגנון XY עם רצועת GT2, בקר ESP32, מנועי DC, מצלמה לזיהוי הדיסקית וחיישני עזר.

הבחירה במנגנון XY נובעת מכך ששולחן ההוקי הוא משטח מלבני, ולכן תנועה בשני צירים מתאימה יותר מתנועה של זרוע רובוטית. בנוסף, מנגנון XY מאפשר שליטה פשוטה יחסית במיקום המגן ומאפשר לרובוט לנוע בתוך חצי השולחן שלו בלבד.

המערכת תורכב מהחלקים העיקריים הבאים:

- שולחן הוקי אוויר קטן.
- מנגנון XY לתנועת המגן.
- שני צירים ליניאריים.
- מוטות 8 מ"מ.
- מיסבי LM8UU.
- תושבות SHF.
- רצועת GT2.
- גלגלות שיניים ו־Idler Pulleys.
- מנועי DC להנעת המנגנון.
- מנוע סרוו לדחיפת הדיסקית.
- בקר ESP32.
- מצלמה לזיהוי מיקום הדיסקית וכיוון תנועתה.
- חיישן מרחק.
- חיישן תאוצה.
- חיישני מגע לזיהוי גולים.
- מסך מגע להצגת ניקוד, מצב משחק והגדרות.
- רמקול ומטריצת לדים לחיווויים במהלך המשחק.

הרכיבים המכניים שנאספו לצורך בניית המנגנון כוללים בין היתר רצועת GT2, גלגלות, מיסבי LM8UU, תושבות SHF8 וקיט ברגי M3.

המערכת נבחרה משום שהיא משלבת מבנה מכני פשוט יחסית, שליטה באמצעות בקר ESP32, זיהוי מצב המשחק באמצעות מצלמה וחיישנים, ויכולת תגובה פיזית של הרובוט בזמן אמת.

9. הסבר על הבקר

שם הבקר: ESP32

בקר ה־ESP32 הוא לוח פיתוח המשמש כיחידת העיבוד המרכזית של מערכת ה־Air Hockey Robot. הבקר נבחר בשל שילוב של כוח עיבוד, קישוריות אלחוטית, גודל פיזי קטן ויכולת עבודה עם מספר חיישנים ואקטואטורים במקביל.

תפקיד הבקר במערכת

ה־ESP32 משמש כ"מוח" של המערכת. הוא אחראי על:

- קבלת נתונים מהמצלמה ומהחיישנים.
- עיבוד בסיסי של מצב המשחק.
- קבלת נקודת XY שאליה יש להזיז את הרובוט.
- שליטה במנועי DC של מנגנון ה־XY.
- הפעלת מנוע סרוו לדחיפת הדיסקית.
- הפעלת מסך המגע, הרמקול ומטריצת הלדים.
- תקשורת עם שרת או מסד נתונים לצורך קוד משחק ושמירת נתונים.
- ניהול מצבי המשחק, הניקוד והזמן.

יכולות עיקריות

לבקר ESP32 יש כמה יתרונות משמעותיים:

- מעבד חזק יחסית למיקרו־בקרים פשוטים.
- תקשורת Wi-Fi מובנית.
- תמיכה ב־Bluetooth.
- מספר רב של פיני GPIO.
- יכולת עבודה עם חיישנים, מנועים ורכיבי תצוגה.
- מעבד כפול ליבה המאפשר ביצוע מספר פעולות במקביל.
- תמיכה ב־ADC לקריאת חיישנים אנלוגיים.
- צריכת חשמל נמוכה יחסית.
- התאמה לפרויקטים של IoT ומערכות משובצות.

זיכרון ועיבוד

ה־ESP32 כולל זיכרון פנימי המאפשר הרצת קוד בקרה, ניהול תקשורת, קריאת חיישנים ושליחת פקודות למנועים. בפרויקט זה הבקר מנהל את פעולת המערכת בזמן אמת, כולל קריאת נתוני המשחק, הפעלת המנועים ועדכון התצוגה.

עיבוד התמונה מתבצע בשרת Python באמצעות OpenCV, וה־ESP32 מקבל מהשרת את נקודת ה־XY להפעלת המנגנון.

תמונה

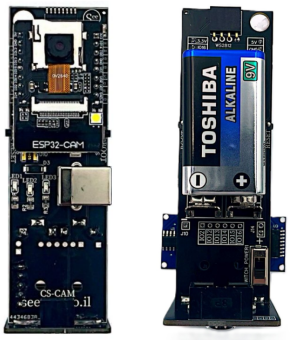


איור: בקר ESP32 — הבקר המרכזי של מערכת רובוט הוקי האוויר.

10. הסבר רכיבים — חיישנים ואופן פעולה

מצלמה / ESP32-CAM

המצלמה היא החיישן המרכזי לזיהוי מיקום הדיסקית. היא מותקנת מעל השולחן בזווית שבה ניתן לראות את אזור המשחק. המצלמה מצלמת את המשטח, והמערכת מנתחת את התמונה כדי לזהות את מיקום הדיסקית ואת כיוון תנועתה.



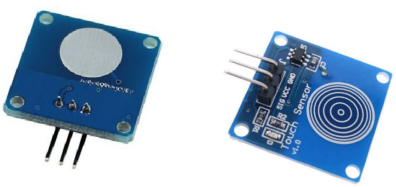
אופן פעולה:

1. צילום תמונה מאזור המשחק.
2. זיהוי הדיסקית בתמונה.
3. חישוב מיקום הדיסקית ביחס לשולחן.
4. חישוב כיוון התנועה של הדיסקית.
5. שליחת הנתונים לאלגוריתם.
6. חישוב נקודת יעד למגן הרובוטי.

חיישן מגע לשער

חיישני המגע משמשים לזיהוי כניסת דיסקית לשער. כאשר הדיסקית פוגעת בחיישן באזור השער, המערכת מזהה שנכבש גול ומעדכנת את הניקוד.

אופן פעולה:

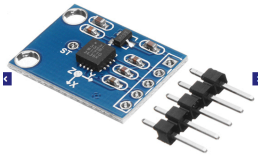


1. הדיסקית מגיעה לאזור השער.
2. החיישן מופעל בעקבות פגיעה או לחיצה.
3. הבקר מזהה אירוע גול.
4. הניקוד מתעדכן במסך המשחק.
5. המערכת מפעילה חיווי מתאים, כגון צליל או תאורה.

חיישן תאוצה / IMU

חיישן התאוצה משמש לזיהוי תנועת היד של השחקן. בפרויקט זה החיישן מותקן על פרק ידו של השחקן, ומסייע להערכת עוצמת ומהירות הדחיפה של הדיסקית.

אופן פעולה:



1. החיישן מודד את תאוצת היד של השחקן.
2. המערכת מזהה שינוי בתנועה בזמן הדחיפה.
3. הנתונים משמשים להערכת עוצמת המכה.
4. המידע מסייע למערכת להבין את מצב המשחק ולשפר את תגובת הרובוט.

חיישן מרחק

חיישן המרחק משמש למדידת המרחק בין השחקן לבין שולחן המשחק. הנתון מסייע למערכת להבין את מיקום השחקן ביחס לשולחן, ולשפר את הערכת מהלך המשחק.

אופן פעולה:



1. החיישן מודד את המרחק מהשחקן.
2. הנתון נשלח לבקר.
3. הבקר משתמש במידע כחלק מנתוני מצב המשחק.
4. המערכת יכולה להתאים את התגובה בהתאם למרחק ולתנועת השחקן.

מנועי DC צהוב

מנועי ה־DC משמשים להנעת מנגנון ה־XY. כל מנוע אחראי על ציר תנועה אחד:



1. מנוע אחד לציר X.
2. מנוע אחד לציר Y.

המנוע מסובב גלגלת GT2, הגלגלת מזיזה את הרצועה, והרצועה גוררת את החלק הנע של המנגנון. כך המגן הרובוטי יכול להגיע לנקודות שונות בחצי השולחן שלו.

דרייבר ULN2003

דרייבר המנועים משמש להפעלת מנועי ה־DC. ה־ESP32 אינו מפעיל את המנועים ישירות, משום שהוא אינו מספק זרם מספיק להפעלתם.



אופן פעולה:

1. ה־ESP32 שולח אותות בקרה לדרייבר.
2. הדרייבר מקבל מתח מתאים למנועים.
3. הדרייבר מפעיל את המנועים לפי כיוון ומהירות שנקבעו בקוד.
4. המנגנון נע בהתאם לפקודות הבקר.

מנוע סרוו

מנוע הסרוו מחובר למנגנון הרובוטי ומשמש לדחיפה מבוקרת של הדיסקית. לאחר שהרובוט מגיע לנקודת המפגש עם הדיסקית, הסרוו משנה זווית ודוחף את הדיסקית לכיוון שטח היריב.



אופן פעולה:

1. הרובוט מגיע לנקודת ה־XY שחושבה.
2. הבקר מפעיל את מנוע הסרוו.
3. הסרוו מזיז את זרוע הדחיפה או חלק המחובר למגן.
4. הדיסקית נדחפת לכיוון שער היריב.
5. הסרוו חוזר למצב ההתחלתי.

רצועת GT2 וגלגלות



הרצועה והגלגלות ממירות את סיבוב המנוע לתנועה ישרה.
גלגלת השיניים מחוברת למנוע, והרצועה נעה יחד איתה.
בצד השני מותקנת גלגלת חופשית, שתפקידה לשמור על מסלול סגור ומתוח של הרצועה.

מסך מגע

המסך משמש להצגת נתוני המשחק ולניהול ההגדרות.

המסך מציג למשתמש:



1. הכנסת קוד משחק.
2. בחירת רמת משחק.
3. בחירת סוג משחק: טיימר או ראשון שמגיע ל־7 נקודות.
4. ניקוד.
5. מצב המשחק.
6. זמן משחק.
7. הודעת ניצחון.

8. מצב מערכת.

רמקול Piezo

הרמקול משמש להשמעת צלילים במהלך המשחק.

לדוגמה:



1. התחלת משחק.
2. גול.
3. סיום משחק.
4. התראת תקלה.
5. הודעת ניצחון.

תאורה

מטריצת הלדים משמשת להצגת חיווי חזותי במהלך המשחק. התאורה עוזרת לשחקן להבין את מצב המשחק בצורה מהירה וברורה.

לדוגמה:



1. אור רגיל במהלך משחק פעיל.
2. אור אדום לאחר גול או בזמן עצירה.
3. אור ירוק כאשר ניתן להתחיל או להמשיך במשחק.
4. חיווי מיוחד בסיום המשחק או בעת ניצחון.

11. תיאור הארכיטקטורה

הארכיטקטורה של מערכת AirHockey Robot בנויה ממספר שכבות הפועלות יחד: שכבת חיישנים, שכבת עיבוד, שכבת בקרה ושכבת ביצוע.

בשלב הראשון, חיישני המערכת אוספים נתונים בזמן אמת משולחן המשחק. המצלמה מזהה את מיקום הדיסקית ואת כיוון תנועתה, חיישן התאוצה מספק מידע על תנועת יד השחקן, חיישן המרחק מודד את מרחק השחקן מהשולחן, וחיישני המגע מזהים כניסת דיסקית לשער.

בשלב השני, תמונת המצלמה מועברת לעיבוד בצד השרת באמצעות Python ו־OpenCV. השרת מנתח את התמונה, מזהה את מיקום הדיסקית, מחשב את כיוון התנועה שלה ואת נקודת ה־XY שאליה הרובוט צריך להגיע.

בשלב השלישי, הנתונים מועברים אל בקר ה־ESP32. הבקר אחראי על קבלת נקודת היעד, שילוב נתוני החיישנים, ניהול מצב המשחק, עדכון הניקוד והפעלת רכיבי החומרה.

בשלב האחרון, ה־ESP32 מפעיל את האקטואטורים המתאימים: מנועי DC להזזת מנגנון ה־XY, מנוע סרוו לדחיפת הדיסקית, רמקול להשמעת צלילים ומטריצת לדים להצגת חיווי חזותי. בנוסף, מסך המגע מציג את קוד המשחק, הגדרות המשחק, הניקוד ומצב המשחק.

מבנה זה יוצר מערכת אוטונומית הפועלת בלולאה סגורה:

חישה ← עיבוד תמונה ← החלטה ← תנועה ← בדיקה מחדש

כך המערכת מזהה את מצב המשחק, מחשבת תגובה מתאימה, מפעילה את הרובוט וממשיכה לעקוב אחר התוצאה בזמן אמת.

12. תיאור פרוטוקולי תקשורת

בפרויקט נעשה שימוש במספר סוגי תקשורת בסיסיים בין בקר ה־ESP32 לבין רכיבי המערכת. כל סוג תקשורת מתאים לרכיב אחר לפי אופן הפעולה שלו.

תקשורת GPIO
תקשורת GPIO משמשת לקריאה והפעלה של רכיבים פשוטים דרך פינים דיגיטליים בבקר.

בפרויקט תקשורת זו משמשת בעיקר ל:

- קריאת חיישני מגע בשערים.
- הפעלת רמקול Piezo.
- קריאת אותות פשוטים מרכיבים חיצוניים.

כאשר חיישן מגע מופעל, מצב הפין משתנה, וה־ESP32 מזהה שנכנס גול ומעדכן את מצב המשחק.

תקשורת PWM
תקשורת PWM משמשת לשליטה ברכיבים שצריכים אות משתנה, ולא רק מצב של דולק או כבוי.

בפרויקט תקשורת זו משמשת ל:

- שליטה במהירות מנועי DC.
- שליטה בזווית מנוע הסרוו.
- אפשרות לשליטה בעוצמת תאורה או חיווי.

באמצעות PWM הבקר יכול לשלוט בצורה מדויקת יותר על פעולת המנועים והרכיבים.

תקשורת I2C
תקשורת I2C משמשת לחיבור רכיבים כמו מסך או חיישן תאוצה לבקר באמצעות שני קווי תקשורת בלבד.

בפרויקט היא יכולה לשמש ל:

- חיישן תאוצה / IMU.

היתרון של I2C הוא חיווט פשוט וחיסכון בפינים של ה־ESP32.

תקשורת Wi-Fi / HTTP
ה־ESP32 כולל Wi-Fi מובנה, ולכן ניתן להשתמש בו לתקשורת עם שרת או אתר המשחק.

בפרויקט תקשורת זו משמשת ל:

- קבלת קוד משחק.
- קבלת הגדרות משחק.
- שליחת מצב משחק או ניקוד.
- תקשורת עם שרת Python לצורך נתוני מיקום או עיבוד.

כאשר נדרש להעביר מידע לשרת, ניתן להשתמש בבקשות HTTP מסוג GET או POST.

13. הסבר ספריות

בפרויקט נעשה שימוש בספריות מוכנות לצד קוד שנכתב במיוחד עבור המערכת.

ספריית WiFi.h משמשת לחיבור בקר ה־ESP32 לרשת אלחוטית, כדי לאפשר תקשורת עם השרת וקבלת נתונים הקשורים למשחק.

ספריית HTTPClient.h משמשת לשליחת בקשות HTTP בין ה־ESP32 לבין השרת, למשל לצורך קבלת קוד משחק והגדרות משחק, שליחת ניקוד או מצב משחק, וקבלת נתוני מיקום או פקודות מהשרת.

בנוסף, ספריית Wire.h משמשת לתקשורת I2C בין ה־ESP32 לבין רכיבים שונים, כגון חיישן תאוצה. תקשורת זו מאפשרת חיבור פשוט יחסית באמצעות שני קווי תקשורת בלבד, ולכן היא מתאימה לרכיבים שדורשים העברת נתונים מסודרת מול הבקר.

מעבר לספריות המוכנות, בפרויקט נכתבים גם קבצי קוד ייעודיים לניהול חלקי המערכת, כגון קוד להפעלת מנועי DC ומנגנון ה־XY, קוד להפעלת מנוע הסרוו, קוד לקריאת חיישנים, קוד לניהול ניקוד ומצבי משחק וקוד לתקשורת עם השרת. עיבוד התמונה מתבצע בצד שרת Python באמצעות OpenCV, ובכך ה־ESP32 יכול להתמקד בעיקר בשליטה על רכיבי החומרה ובניהול מהלך המשחק.

14. תהליך הפיתוח

תהליך הפיתוח התחיל מבחירת רעיון הפרויקט: רובוט הוקי אוויר המשחק מול אדם. בתחילה נבדקו כמה חלופות מכניות, כמו זרוע רובוטית, מנגנון מגנטי ומנגנון XY. לאחר בדיקה נבחר מנגנון XY, משום שהוא מתאים יותר לשולחן מלבני ומאפשר תנועה ברורה בשני צירים.

לאחר מכן נבחרו רכיבי המערכת: שולחן הוקי קטן, מוטות ליניאריים, מיסבים, תושבות, רצועת GT2, גלגלות, מנועי DC, מנוע סרוו, בקר ESP32 וחיישנים. בבחירת הרכיבים היה צורך להתחשב במחיר, בגודל השולחן ובכך שהמערכת מיועדת לדגם לימודי.

בשלב הבנייה המכני עלו כמה קשיים. היה צורך לקבע את המוטות לשולחן קטן, לשמור על מקביליות בין הצירים ולוודא שהתנועה חלקה. בנוסף היה צריך למצוא דרך לחבר את הרצועה ליחידה הנעה בצורה יציבה, כך שתוכל להזיז את המגן בלי החלקות או תזוזות לא רצויות.

לאחר סיום החלק המכני הגיע שלב חיבור מנועי ה־DC לבקר ולדרייבר המנועים. בשלב זה היה צורך לבדוק שכל ציר זז בחופשיות, משום שכל חיכוך מכני עלול להפריע לתנועה ולפגוע בדיוק של מנגנון ה־XY. בהמשך שולבו החיישנים, המצלמה, מסך המגע ורכיבי החיווי, ונבנה האלגוריתם שמחשב את מיקום המגן ואת תגובת הרובוט לפי מצב המשחק.

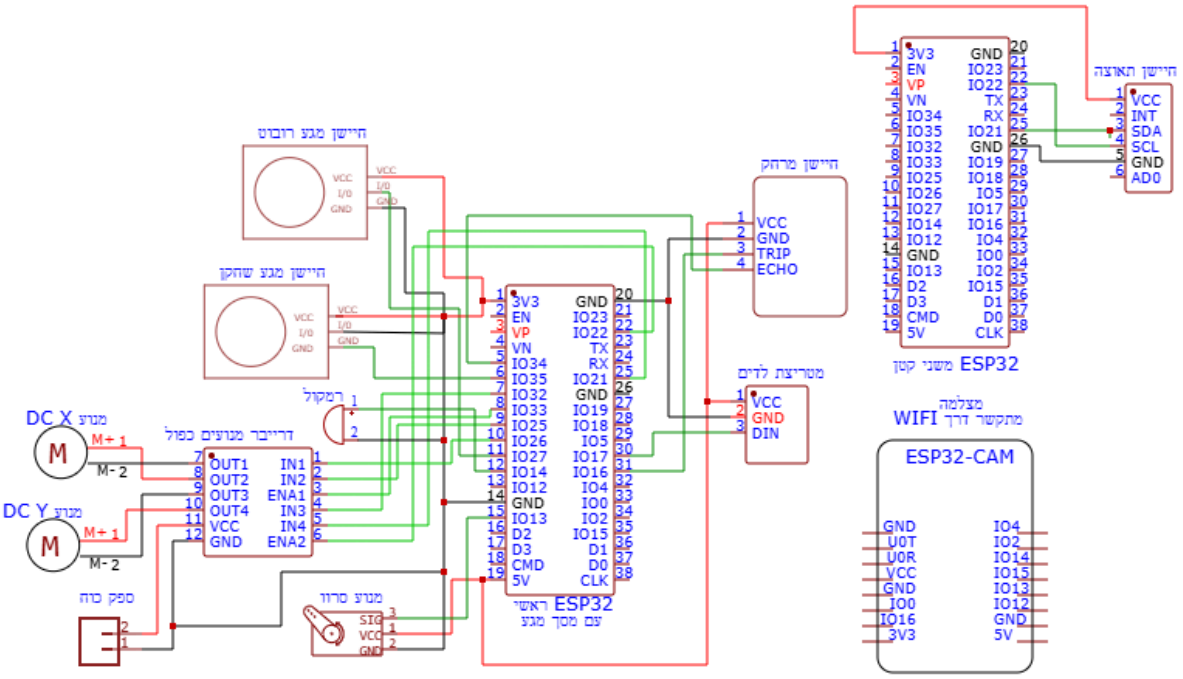
15. תיעוד הפתרון

a. שרטוט חשמלי:

המערכת החשמלית מבוססת על בקר ESP32, המשמש כבקר המרכזי של הפרויקט. אל הבקר מחוברים רכיבי הקלט והפלט של המערכת: מצלמה, חיישנים, מנועים, מסך מגע, רמקול ומטריצת לדים.

המנועים אינם מחוברים ישירות לבקר, מכיוון שה־ESP32 אינו מסוגל לספק להם את הזרם הדרוש להפעלה ישירה. לכן מנועי ה־DC מחוברים דרך דרייבר מנועים מתאים, וה־ESP32 שולח לדרייבר פקודות כיוון ומהירות. בנוסף, מנוע הסרוו מחובר לבקר ומשמש לדחיפת הדיסקית לאחר שהרובוט הגיע לנקודת המפגש.

מבנה החיבורים הכללי:



בתרשים מוצג ESP32 כללי לצורך המחשת החיבורים. בפועל נעשה שימוש בלוח ESP32 עם מסך מגע.

b. חיבורים ואופן פעולה:

החיבורים במערכת מבוססים על בקר ESP32, המחובר לחיישנים, מנועים ורכיבי חיווי. לכל רכיב מוקצים פינים מתאימים בבקר, ויש להקפיד שלא להשתמש באותו GPIO עבור כמה רכיבים שונים. בנוסף, כל רכיבי המערכת חייבים להיות מחוברים ל־GND משותף, כדי שהאותות החשמליים יזוהו בצורה תקינה.

מנוע DC למנגנון XY:

דרייבר מנועים לציר Y-X	לבקר
IN1	GPIO 26
IN2	GPIO 25
PWM	GPIO 33
IN3	GPIO 32
IN4	GPIO 21
PWM	GPIO 22
VCC	5V
GND	GND
OUT1	M- מנוע X
OUT2	M+ מנוע X
OUT3	M- מנוע Y
OUT4	M+ מנוע Y

אופן פעולה: ה־ESP32 שולח לדרייבר אותות כיוון ומהירות. הדרייבר מפעיל את מנועי ה־DC, וכל מנוע מזיז ציר אחר במנגנון ה־XY.

מנוע סרוו:

מחיישן המגע	לבקר
VCC	5V
GND	GND
SIG	GPIO 13

אופן פעולה: ה־ESP32 שולח לסרוו אות PWM, והסרוו משנה זווית כדי לבצע דחיפה מבוקרת של הדיסקית.

חיישן מגע:

מחיישן המגע	לבקר
VCC	3V3
GND	GND
SIG	בשער השחקן - GPIO 35 , בשער הרובוט - GPIO 27

אופן פעולה: כאשר הדיסקית מפעילה את החיישן, מצב הפין משתנה, וה־ESP32 מזהה שנכנס גול ומעדכן את הניקוד.

מטריצת לדים:

מהתאורה	לבקר
4-7VDC / VCC	5V
GND	GND 5 שלV הספק
DIN	GPIO 17

אופן פעולה: ה־ESP32 שולח אות נתונים לפין DI, והמטריצה מציגה חיווי צבעוני לפי מצב המשחק.

רמקול:

מהרמקול	לבקר
אדום Signal	GPIO 14
שחור GND	GND

אופן פעולה: ה־ESP32 מפעיל את הרמקול להשמעת צלילים בזמן התחלת משחק, גול, סיום משחק או תקלה.

חיישן תאוצה:

מחיישן התאוצה	לבקר ESP הקטן - המשני
VCC	3V3
GND	GND
SDA	GPIO 21

SCL	GPIO 22

אופן פעולה: החיישן מתקשר עם ה־ESP32 בתקשורת I2C. הוא מודד את תנועת יד השחקן ומסייע בהערכת עוצמת ומהירות הדחיפה.

חיישן מרחק:

מחיישן המרחק	לבקר
VCC	5V
GND	GND
Trig	GPIO 16
Echo	דרך מחלק מתח GPIO 34

אופן פעולה: ה־ESP32 שולח פולס דרך Trig, החיישן מחזיר אות דרך Echo, ומהזמן שעבר ניתן לחשב את המרחק.

c. תיעוד קטעי קוד עיקריים כולל הסברים:

אתחול המערכת

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("========== ESP32 START ==========");
  Serial.println("BOOT START");

  lcd_init();
  lcd_clear();
  lcd_print_text(NUM_20, NUM_20, "Booting...");
  Serial.println("LCD initialized");

  touch_init();
  Serial.println("Touch screen initialized");

  connect();
  Serial.println("WiFi connected");
```

```
  init_time();
  Serial.println("Time initialized");

  init_firebase();
  Serial.println("Firebase initialized");

  init_react_server();
  Serial.println("React server initialized");

  init_dc_motors();
  Serial.println("DC motors initialized");

  init_servo();
  Serial.println("Servo initialized");

  init_led();
  Serial.println("LED initialized");

  init_speaker();
  Serial.println("Speaker initialized");

  init_touch_Sensor();
  Serial.println("Goal touch sensor initialized");

  init_accel();
  Serial.println("MPU6050 initialized");

  init_distance_Sensor();
  Serial.println("Distance sensor initialized");

  lcd_clear();
  lcd_print_text(NUN_20, NUM_20, "ESP32 Ready");
  Serial.println("ESP32 Ready");

  delay(1000);

  showEnterCodeScreen();
  Serial.println("Enter code screen shown");
}
```

פונקציה זו מופעלת פעם אחת בעת הדלקת ה־ESP32. היא מאתחלת את כל רכיבי המערכת: מסך LCD, מסך מגע, חיבור Wi-Fi, זמן מערכת, Firebase, שרת מקומי, מנועי DC, מנוע סרוו, מטריצת לדים, רמקול, חיישן מגע, חיישן תאוצה וחיישן מרחק. בסיום האתחול מוצג במסך שהמערכת מוכנה, ולאחר מכן נפתח מסך הכנסת קוד המשחק.

לולאת העבודה הראשית

```
void loop() {
  handle_react_server();
  switch (currentScreen) {
    case SCREEN_ENTER_CODE:
      handleEnterCodeScreen();
      break;
    case SCREEN_CHECKING_CODE:
      break;
    case SCREEN_GAME_FOUND:
      handleGameFoundScreen();
      break;
    case SCREEN_SELECT_LEVEL:
      handleSelectLevelScreen();
      break;
    case SCREEN_SELECT_MODE:
      handleSelectModeScreen();
      break;
    case SCREEN_READY_TO_START:
      handleReadyToStartScreen();
      break;
    case SCREEN_LIVE_GAME:
      runLiveGameLogic();
      break;
    case SCREEN_ERROR:
      handleErrorScreen();
      break;
    default:
      break;
  }
}
```

פונקציה זו רצה כל הזמן לאחר סיום האתחול. בכל מחזור היא מטפלת בבקשות שמגיעות מהשרת או מהאתר, ולאחר מכן בודקת באיזה מצב מסך נמצאת המערכת. בהתאם למצב הנוכחי היא מפעילה את הטיפול המתאים: הכנסת קוד, בחירת רמה, בחירת סוג משחק, התחלת משחק, משחק פעיל או מסך שגיאה.

ניהול המשחק בזמן אמת

```
void runLiveGameLogic() {
  updateLiveGameScreen();

  // אורות צבעוניים רצים במהלך המשחק
  ledGameActiveEffect();
  // בדיקת חיישן מרחק - בטיחות / עבירת משחק
  float distanceCm = getDistanceCm();
```

```
if (distanceCm > 0 && distanceCm < MIN_SAFE_DISTANCE_CM) {
  stopMotors();
  lcd_clear();
  lcd_print_text(NUM_20, NUM_40, "Safety Stop");
  lcd_print_text(NUM_20, NUM_80, "Too close!");
  ledViolationEffect();
  speakerViolation();
  delay(700);
  showLiveGameScreen();
  return;
}

// בדיקת חיישן מגע - גול
if (isTouchDetected() && millis() - lastGoalMillis > GOAL_COOLDOWN_MS) {
  lastGoalMillis = millis();
  player1Score++;
  lcd_clear();
  lcd_print_text(NUM_20, NUM_40, "Goal!");
  lcd_print_text(NUM_20, NUM_80, "Player +1");
  ledGoalEffect();
  speakerGoal();

  // אחרי גול: עצירה באדום
  ledSetAllRed();
  delay(1200);

  ledSetAllGreen();
  delay(1200);

  gameTimeSeconds = (millis() - gameStartMillis) / MILLIS;

  send_game_data(
    tableNumber,
    player1Score,
    player2Score,
    puckSpeed,
    gameTimeSeconds,
    playerName
  );

  if (selectedGameMode == MODE_FIRST_TO_7 && player1Score >= MAX_GOL) {
    finishLiveGame();
    return;
  }
  showLiveGameScreen();
  return;
}
```

```
  // קריאת חיישן תאוצה
  sensors_event_t acceleration, gyro, temperature;
  mpu.getEvent(&acceleration, &gyro, &temperature);
  bool puckMoved = isPuckMoving(
    acceleration.acceleration.x,
    acceleration.acceleration.y,
    acceleration.acceleration.z
  );
  gameTimeSeconds = (millis() - gameStartMillis) / MILLIS;

  // סיום לפי טיימר
  if (selectedGameMode == MODE_TIMER_3_MINUTES) {
    if (gameTimeSeconds >= 180) {
      finishLiveGame();
      return;
    }
  }

  // סיום לפי 7 נקודות
  if (selectedGameMode == MODE_FIRST_TO_7) {
    if (player1Score >= MAX_GOL || player2Score >= MAX_GOL) {
      finishLiveGame();
      return;
    }
  }

  // עדכון Firebase אחת לכמה שניות
  if (millis() - lastFirebaseUpdate > NUMBER_UPDATE) {
    send_game_data(
      tableNumber,
      player1Score,
      player2Score,
      puckSpeed,
      gameTimeSeconds,
      playerName
    );

    lastFirebaseUpdate = millis();
  }
}
```

פונקציה זו מנהלת את מהלך המשחק בפועל. היא מעדכנת את מסך המשחק, מפעילה אורות צבעוניים בזמן המשחק, בודקת את חיישן המרחק, מזהה גול באמצעות חיישן המגע, מעדכנת ניקוד, מפעילה אפקט לדים ורמקול בזמן גול, בודקת האם המשחק הסתיים לפי זמן או לפי 7 נקודות, ושולחת עדכוני משחק ל־Firebase.

חיבור לרשת

```
void connect() {
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connected!");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
}
```

הפונקציה מחברת את בקר ה־ESP32 לרשת Wi-Fi. לאחר ההתחברות היא מציגה במסך הסריאלי את כתובת ה־IP של הבקר, כדי שיהיה ניתן לתקשר איתו מהשרת או מהאתר.

אתחול FireBase

```
void init_firebase() {
  config.api_key = API_KEY;
  config.timeout.serverResponse = NUM_RESPONSE;
  config.timeout.wifiReconnect = NUM_RESPONSE;
  auth.user.email = USER_EMAIL;
  auth.user.password = USER_PASSWORD;
  config.token_status_callback = tokenStatusCallback;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
  while (!Firebase.ready()) {
    Serial.println("Waiting for Firebase...");
    delay(1000);
  }
  fbdo.setBSSLBufferSize(8192, 2048);
  Serial.println("Firebase Ready!");
}
```

הפונקציה מאתחלת את החיבור ל־Firebase. היא מגדירה את פרטי ההתחברות, מפעילה את השירות וממתינה עד שהחיבור מוכן. דרך Firebase המערכת מקבלת קודי משחק ושומרת נתוני משחק.

שליחת נתוני משחק

```
void send_game_data(int table, int p1, int p2, int speed, int timeSec, const
String& playerName) {
  FirebaseJson content;

  content.set("fields/id/stringValue", "game1");
  content.set("fields/table_number/integerValue", table);
  content.set("fields/player1_score/integerValue", p1);
  content.set("fields/player2_score/integerValue", p2);
  content.set("fields/puck_speed/integerValue", speed);
  content.set("fields/game_time_seconds/integerValue", timeSec);
  content.set("fields/opponent_type/stringValue", "robot");
  content.set("fields/opponent_name/stringValue", "Robot");

  if (p1 >= MAX_GOL || p2 >= MAX_GOL) {
    content.set("fields/status/stringValue", "completed");
    content.set("fields/is_active/booleanValue", false);

    if (p1 >= MAX_GOL) {
      content.set("fields/winner/stringValue", playerName);
      content.set("fields/last_goal_by/stringValue", "player1");
    } else {
      content.set("fields/winner/stringValue", "Robot");
      content.set("fields/last_goal_by/stringValue", "player2");
    }
  } else {
    content.set("fields/status/stringValue", "in_progress");
    content.set("fields/is_active/booleanValue", true);

    if (p1 > p2) {
      content.set("fields/last_goal_by/stringValue", "player1");
    } else if (p2 > p1) {
      content.set("fields/last_goal_by/stringValue", "player2");
    } else {
      content.set("fields/last_goal_by/stringValue", "none");
    }
    content.set("fields/winner/stringValue", "");
  }

  content.set("fields/updated_at/stringValue", String((unsigned
long)time(nullptr)));

  String documentPath = "ESP32Data/game1";

  if (Firebase.Firestore.patchDocument(
        &fbdo,
        FIREBASE_PROJECT_ID,
        DATABASE_ID,
        documentPath.c_str(),
```

```
        content.raw(),

"id,table_number,player1_score,player2_score,puck_speed,game_time_seconds,opp
onent_type,opponent_name,status,is_active,last_goal_by,winner,updated_at"
      )) {
    Serial.println("Game data sent!");
  } else {
    Serial.println(fbdo.errorReason());
  }
}
```

הפונקציה שולחת ל־Firebase את נתוני המשחק: מספר שולחן, ניקוד השחקנים, מהירות דיסקית, זמן משחק, מצב המשחק והמנצח במקרה שהמשחק הסתיים. כך ניתן לעקוב אחר מצב המשחק גם מחוץ ל־ESP32.

אתחול השרת המקומי

```
void init_react_server() {

  server.on("/game", HTTP_GET, handleGetGame);
  server.on("/game", HTTP_POST, handlePostGame);
  server.on("/game", HTTP_OPTIONS, sendOptions);
  server.on("/game-by-code", HTTP_GET, handleGetGameByCode);
  server.on("/game-by-code", HTTP_OPTIONS, sendOptions);
  server.on("/start-game", HTTP_POST, handleStartGame);
  server.on("/start-game", HTTP_OPTIONS, sendOptions);
  server.on("/subscriptions", HTTP_GET, handleGetSubscriptions);
  server.on("/subscriptions", HTTP_POST, handlePostSubscriptions);
  server.on("/subscriptions", HTTP_OPTIONS, sendOptions);
  server.on("/subscription", HTTP_PUT, handlePutSubscription);
  server.on("/subscription", HTTP_DELETE, handleDeleteSubscription);
  server.on("/subscription", HTTP_OPTIONS, sendOptions);
  server.on("/user-subscription", HTTP_GET, handleGetUserSubscription);
  server.on("/user-subscription", HTTP_OPTIONS, sendOptions);
  server.on("/user-bonuses", HTTP_GET, handleGetUserBonuses);
  server.on("/user-bonuses", HTTP_OPTIONS, sendOptions);
  server.on("/subscribe", HTTP_POST, handlePostSubscribe);
  server.on("/subscribe", HTTP_OPTIONS, sendOptions);
  server.on("/home-data", HTTP_GET, handleGetHomeData);
  server.on("/history", HTTP_GET, handleGetHistory);
  server.on("/leaderboard", HTTP_GET, handleGetLeaderboard);
  server.on("/admin-dashboard", HTTP_GET, handleGetAdminDashboard);
  server.on("/players", HTTP_GET, handleGetPlayers);
  server.on("/high-scores", HTTP_GET, handleGetHighScores);
  server.on("/bonuses", HTTP_GET, handleGetBonuses);
  server.on("/bonuses", HTTP_POST, handlePostBonus);
  server.on("/games", HTTP_GET, handleGetGamesByDate);
```

```
  server.on("/login", HTTP_POST, handleLogin);
  server.on("/login", HTTP_OPTIONS, sendOptions);
  server.on("/register", HTTP_POST, handleRegister);
  server.on("/register", HTTP_OPTIONS, sendOptions);
  server.on("/bonus", HTTP_DELETE, handleDeleteBonus);
  server.on("/bonus", HTTP_OPTIONS, sendOptions);
  server.on("/game-item", HTTP_GET, handleGetGameItem);
  server.on("/game-item", HTTP_PUT, handlePutGame);
  server.on("/game-item", HTTP_DELETE, handleDeleteGame);
  server.on("/game-item", HTTP_OPTIONS, sendOptions);
  server.on("/move", HTTP_GET, handleMoveCommand);
  server.on("/move", HTTP_OPTIONS, sendOptions);
  server.begin();
  Serial.println("React bridge server started!");
}
```

הפונקציה מגדירה את השרת המקומי שרץ על ה־ESP32. באמצעות שרת זה האתר או שרת העיבוד יכולים לשלוח לבקר בקשות, כמו קבלת מצב משחק או פקודת תנועה לרובוט.

תנועת הרובוט ומנגנון XY

```
void handle_react_server() {
  server.handleClient();
}
```

הפונקציה מטפלת בבקשות שמגיעות לשרת המקומי של ה־ESP32. היא נקראת שוב ושוב מתוך loop, כדי שהבקר יוכל להגיב בזמן אמת לבקשות מהאתר או מהשרת.

קבלת פקודת תנועה

```
void handleMoveCommand() {
  if (server.method() == HTTP_OPTIONS) {
    sendOptions();
    return;
  }

  if (!server.hasArg("x") || !server.hasArg("y")) {
    sendJson(400, "{\"success\":false,\"message\":\"missing_x_or_y\"}");
    return;
  }

  float x = server.arg("x").toFloat();
  float y = server.arg("y").toFloat();
  Serial.print("Move request from image server: X=");
```

```
  Serial.print(x);
  Serial.print(" Y=");
  Serial.println(y);
  moveToPositionCm(x, y);
  // דחיפה עם הסרוו – אחרי הגעה לנקודת XY
  hitPuck();
  String response = "{";
  response += "\"success\":true,";
  response += "\"message\":\"robot_moved\",";
  response += "\"x\":" + String(x, 2) + ",";
  response += "\"y\":" + String(y, 2);
  response += "}";
  sendJson(200, response);
}
```

הפונקציה מקבלת מהשרת ערכי x ו־y, שהם נקודת היעד שאליה הרובוט צריך להגיע. אם הערכים תקינים, היא מזיזה את הרובוט לנקודה המתאימה בעזרת moveToPositionCm, ולאחר מכן מפעילה את הסרוו לדחיפת הדיסקית.

הזזה לנקודת יעד

```
void moveToPositionCm(float xCm, float yCm) {
  xCm = constrain(xCm, MIN_X_CM, MAX_X_CM);
  yCm = constrain(yCm, MIN_Y_CM, MAX_Y_CM);

  float dx = xCm - currentX;
  float dy = yCm - currentY;
  int dirX = dx >= 0 ? 1 : -1;
  int dirY = dy >= 0 ? 1 : -1;
  unsigned long moveTimeX = abs(dx) * CM_TO_TIME_MS_X;
  unsigned long moveTimeY = abs(dy) * CM_TO_TIME_MS_Y;

  Serial.print("Move request: X=");
  Serial.print(xCm);
  Serial.print(" Y=");
  Serial.println(yCm);

  unsigned long startTime = millis();

  if (moveTimeX > 0) {
    setMotorX(dirX, DEFAULT_MOTOR_SPEED);
  }
  if (moveTimeY > 0) {
    setMotorY(dirY, DEFAULT_MOTOR_SPEED);
  }
```

```
  while ((millis() - startTime < moveTimeX) || (millis() - startTime <
moveTimeY)) {
    unsigned long elapsed = millis() - startTime;

    if (elapsed >= moveTimeX) {
      stopMotorX();
    }
    if (elapsed >= moveTimeY) {
      stopMotorY();
    }
    handle_react_server();
    delay(1);
  }

  stopMotors();
  currentX = xCm;
  currentY = yCm;
}
```

הפונקציה מקבלת נקודת יעד בסנטימטרים בצירי X ו־Y. היא מגבילה את הערכים לתחום השולחן, מחשבת את המרחק מהמיקום הנוכחי, קובעת כיוון לכל ציר ומפעילה את מנועי ה־DC למשך זמן מחושב. בסיום התנועה המנועים נעצרים והמיקום החדש נשמר.

אתחול מנועי DC

```
void init_dc_motors() {
  pinMode(X_IN1, OUTPUT);
  pinMode(X_IN2, OUTPUT);
  pinMode(Y_IN1, OUTPUT);
  pinMode(Y_IN2, OUTPUT);
  ledcSetup(MOTOR_X_FORWARD_CHANNEL, MOTOR_PWM_FREQ, MOTOR_PWM_RESOLUTION);
  ledcSetup(MOTOR_X_BACKWARD_CHANNEL, MOTOR_PWM_FREQ, MOTOR_PWM_RESOLUTION);
  ledcSetup(MOTOR_Y_FORWARD_CHANNEL, MOTOR_PWM_FREQ, MOTOR_PWM_RESOLUTION);
  ledcSetup(MOTOR_Y_BACKWARD_CHANNEL, MOTOR_PWM_FREQ, MOTOR_PWM_RESOLUTION);
  ledcAttachPin(X_IN1, MOTOR_X_FORWARD_CHANNEL);
  ledcAttachPin(X_IN2, MOTOR_X_BACKWARD_CHANNEL);
  ledcAttachPin(Y_IN1, MOTOR_Y_FORWARD_CHANNEL);
  ledcAttachPin(Y_IN2, MOTOR_Y_BACKWARD_CHANNEL);
  stopMotors();
}
```

פונקציה זו מאתחלת את מנועי ה־DC של מנגנון ה־XY. היא מגדירה את פיני המנועים כפלטים, מגדירה ערוצי PWM לשליטה במהירות, ומוודאת שבתחילת הפעולה כל המנועים כבויים.

הפעלת מנועי X ו-Y

```
void setMotorX(int direction, int speedValue) {
  speedValue = constrain(speedValue, 0, 255);

  if (direction > 0) {
    ledcWrite(MOTOR_X_FORWARD_CHANNEL, speedValue);
    ledcWrite(MOTOR_X_BACKWARD_CHANNEL, 0);
  } else if (direction < 0) {
    ledcWrite(MOTOR_X_FORWARD_CHANNEL, 0);
    ledcWrite(MOTOR_X_BACKWARD_CHANNEL, speedValue);
  } else {
    stopMotorX();
  }
}
```

```
void setMotorY(int direction, int speedValue) {
  speedValue = constrain(speedValue, 0, 255);

  if (direction > 0) {
    ledcWrite(MOTOR_Y_FORWARD_CHANNEL, speedValue);
    ledcWrite(MOTOR_Y_BACKWARD_CHANNEL, 0);
  } else if (direction < 0) {
    ledcWrite(MOTOR_Y_FORWARD_CHANNEL, 0);
    ledcWrite(MOTOR_Y_BACKWARD_CHANNEL, speedValue);
  } else {
    stopMotorY();
  }
}
```

שתי הפונקציות מפעילות את מנועי ה־DC של מנגנון ה־XY. כל פונקציה מקבלת כיוון ומהירות: setMotorX שולטת בציר X, ו־setMotorY שולטת בציר Y. בעזרתן הרובוט יכול לנוע ימינה, שמאלה, קדימה ואחורה.

עצירת המנועים

```
void stopMotors() {
  stopMotorX();
  stopMotorY();
}
```

פונקציה זו עוצרת את שני מנועי ה־DC של מנגנון ה־XY. היא משמשת בסיום תנועה, במצב בטיחות, ובכל מצב שבו צריך לעצור את הרובוט מיד.

דחיפת הדיסקית

```
void hitPuck() {
  hitServo.write(SERVO_HIT_ANGLE);
  delay(SERVO_HIT_DELAY_MS);
  hitServo.write(SERVO_REST_ANGLE);
}
```

הפונקציה מפעילה את הסרוו לזווית מוגדרת לזמן קצר, ולאחר מכן מחזירה אותו למצב ההתחלתי. פעולה זו משמשת לדחיפת הדיסקית לכיוון שטח היריב.

זיהוי גול

```
bool isTouchDetected() {
  return digitalRead(TOUCH_SENSOR_PIN) == HIGH;
}
```

הפונקציה קוראת את מצב חיישן המגע. כאשר מתקבל ערך פעיל, המערכת מזהה שנכנס גול, מעדכנת ניקוד ומפעילה חיווי מתאים במסך, בלדים וברמקול.

קריאת מרחק

```
float getDistanceCm() {
  digitalWrite(DIST_TRIG_PIN, LOW);
  delayMicroseconds(NUM_2);

  digitalWrite(DIST_TRIG_PIN, HIGH);
  delayMicroseconds(NUM_10);

  digitalWrite(DIST_TRIG_PIN, LOW);

  long duration = pulseIn(DIST_ECHO_PIN, HIGH, NUM_RESPONSE);

  if (duration == 0) {
    return -1;
  }

  return duration * 0.0343 / 2;
}
```

הפונקציה getDistanceCm קוראת את חיישן המרחק ומחזירה את המרחק שנמדד בסנטימטרים. המערכת משתמשת במדידות המרחק כדי לקבל מידע נוסף על מצב המשחק, ובפרט כדי לסייע בהערכת מהירות

הדיסקית או עוצמת הדחיפה שלה. הנתון משתלב עם נתוני המצלמה וחיישן התאוצה לצורך חישוב תגובת הרובוט.

זיהוי תנועה לפי תאוצה

```
bool isPuckMoving(float ax, float ay, float az) {
  float totalAcceleration = abs(ax) + abs(ay) + abs(az);

  if (totalAcceleration > ACCEL_MOVEMENT_THRESHOLD) {
    return true;
  }

  return false;
}
```

הפונקציה מקבלת ערכי תאוצה בצירי X, Y ו־Z ובודקת האם קיימת תנועה משמעותית. הנתון מסייע למערכת להבין האם הייתה תנועה או דחיפה משמעותית במהלך המשחק.

בדיקת קוד משחק

```
void checkGameCodeFromScreen() {
  String foundDocId = "";
  FirebaseJson foundDoc;

  Serial.println("========== CHECK GAME CODE ==========");
  Serial.print("Entered code: ");
  Serial.println(enteredGameCode);

  bool found = loadGameByCode(enteredGameCode, foundDocId, foundDoc);

  Serial.print("Found: ");
  Serial.println(found ? "YES" : "NO");
  Serial.print("Found doc id: ");
  Serial.println(foundDocId);

  if (!found) {
    screenErrorMessage = "Game not found";
    currentScreen = SCREEN_ERROR;
    showErrorScreen();
    return;
  }

  String status = firestoreStringField(foundDoc, "status", "scheduled");
  status.trim();
```

```
  status.toLowerCase();

  Serial.print("Game status: ");
  Serial.println(status);

  if (status != "scheduled") {
    screenErrorMessage = "Game already used";
    currentScreen = SCREEN_ERROR;
    showErrorScreen();
    return;
  }

  selectedGameId = foundDocId;
  selectedGameCode = firestoreStringField(foundDoc, "game_code",
enteredGameCode);

  if (selectedGameCode == "") {
    selectedGameCode = enteredGameCode;
  }

  currentScreen = SCREEN_GAME_FOUND;
  showGameFoundScreen();
}
```

הפונקציה בודקת את קוד המשחק מול Firebase. אם נמצא משחק מתאים, המערכת ממשיכה לבחירת רמה וסוג משחק. אם הקוד לא נמצא, מוצגת הודעת שגיאה.

התחלת משחק

```
bool startSelectedGame() {
  if (selectedGameId == "") {
    return false;
  }
  String documentPath = "games/" + selectedGameId;
  FirebaseJson existingGame;
  if (!Firebase.Firestore.getDocument(
        &fbdo,
        FIREBASE_PROJECT_ID,
        DATABASE_ID,
        documentPath.c_str()
      )) {
    return false;
  }

  existingGame.setJsonData(fbdo.payload());
```

```
  String currentStatus = firestoreStringField(existingGame, "status",
"scheduled");
  currentStatus.trim();
  currentStatus.toLowerCase();

  if (currentStatus != "scheduled") {
    return false;
  }

  FirebaseJson content;
  content.set("fields/status/stringValue", "in_progress");
  content.set("fields/skill_level/integerValue", selectedLevel);
  content.set("fields/game_mode/stringValue", selectedModeToString());
  content.set("fields/game_code/stringValue", selectedGameCode);
  content.set("fields/player1_score/integerValue", 0);
  content.set("fields/player2_score/integerValue", 0);
  content.set("fields/winner/stringValue", "");

  bool ok = Firebase.Firestore.patchDocument(
    &fbdo,
    FIREBASE_PROJECT_ID,
    DATABASE_ID,
    documentPath.c_str(),
    content.raw(),

"status,skill_level,game_mode,game_code,player1_score,player2_score,winner"
  );

  if (ok) {
    playerScore = 0;
    robotScore = 0;
    player1Score = 0;
    player2Score = 0;
    puckSpeed = 0;
    gameTimeSeconds = 0;
    gameStartMillis = millis();
    lastFirebaseUpdate = 0;
    lastGoalMillis = 0;
  }

  return ok;
}
```

הפונקציה מעדכנת את המשחק ב־Firebase ממצב מתוכנן למצב פעיל. היא שומרת את רמת המשחק וסוג המשחק שנבחרו, מאפסת ניקוד וזמן, ומכינה את המערכת לתחילת משחק.

סיום משחק

```
void finishLiveGame() {
  stopMotors();
  ledGameOverEffect();
  speakerGameOver();

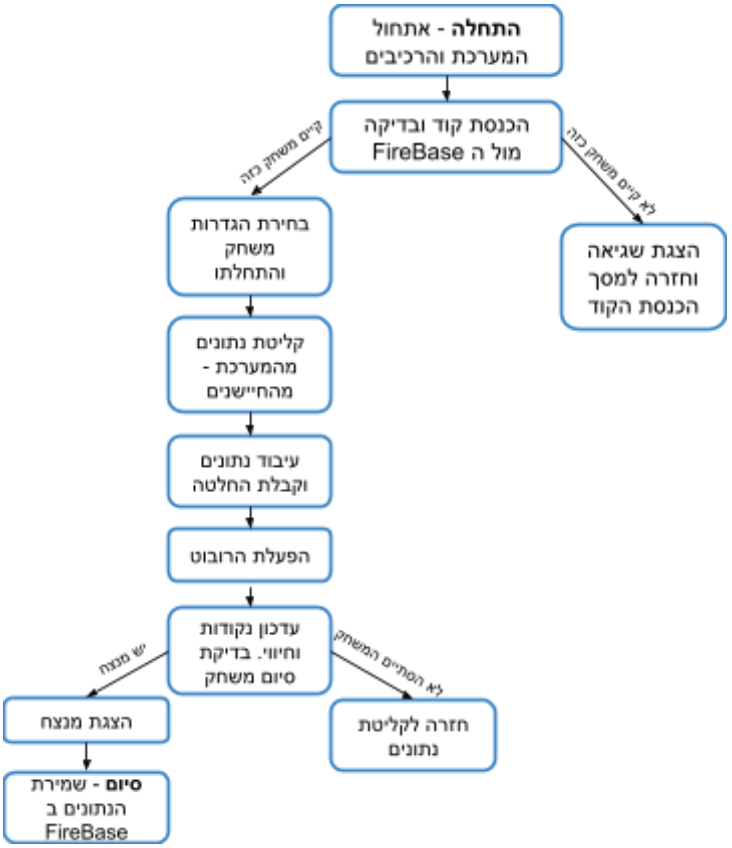
  String winner = "";

  if (player1Score > player2Score) {
    winner = "Player";
  } else if (player2Score > player1Score) {
    winner = "Robot";
  } else {
    winner = "Draw";
  }
  if (selectedGameId != "") {
    String documentPath = "games/" + selectedGameId;

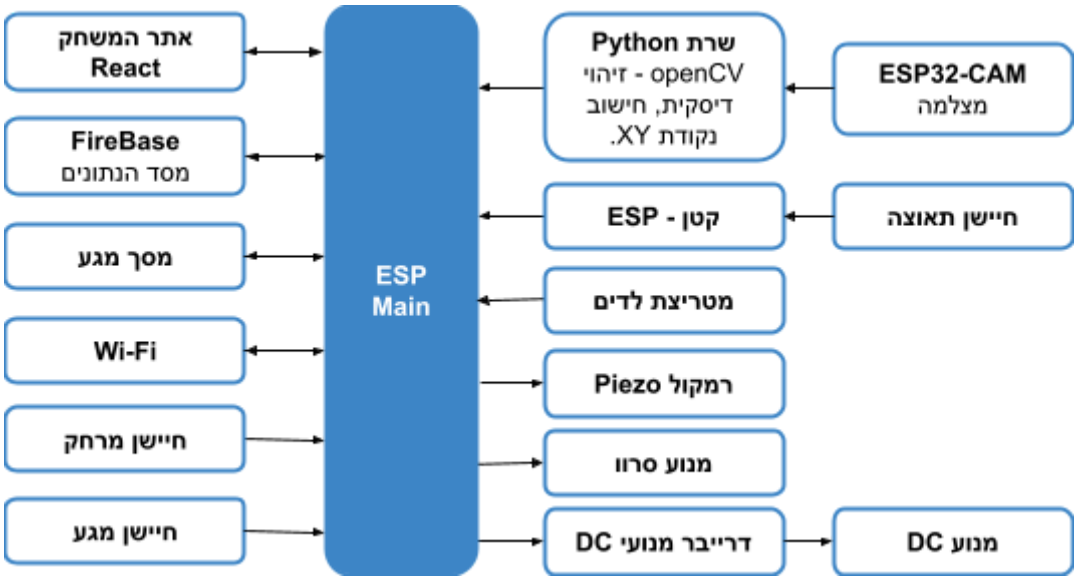
    FirebaseJson content;
    content.set("fields/status/stringValue", "completed");
    content.set("fields/player1_score/integerValue", player1Score);
    content.set("fields/player2_score/integerValue", player2Score);
    content.set("fields/winner/stringValue", winner);
    Firebase.Firestore.patchDocument(
      &fbdo,
      FIREBASE_PROJECT_ID,
      DATABASE_ID,
      documentPath.c_str(),
      content.raw(),
      "status,player1_score,player2_score,winner"
    );
  }
  clearGameScreen();
  drawRoundPanel(10, 20, 300, 195);
  drawText2(92, 48, "Game Over", UI_WHITE);
  drawText2(45, 95, "Winner: " + winner, UI_GREEN);
  drawText2(45, 130, "P: " + String(player1Score) + " R: " +
String(player2Score), UI_WHITE);
  drawGradientButton(NUM_45, NUM_170, NUM_230, NUM_38, "New Game");
  currentScreen = SCREEN_ERROR;
  screenErrorMessage = "Game completed";
}
```

הפונקציה מסיימת את המשחק. היא עוצרת את המנועים, מפעילה אפקט סיום בלדים וברמקול, מחשבת את המנצח, מעדכנת את Firebase ומציגה במסך הודעת סיום.

d. תרשים זרימה:



e. תרשים פונקציונאלי:



f. התייחסות למערכת הפעלה:

המערכת אינה מבוססת על מערכת הפעלה מלאה, אלא על בקרי ESP32 המריצים קוד ייעודי בסביבת Arduino. ה־ESP32 הראשי אחראי על ניהול המשחק, קריאת חיישנים, הפעלת מנועים, מסך, לדים ורמקול.

בצד עיבוד התמונה נעשה שימוש במחשב/שרת המריץ Python ו־OpenCV, ושם מתבצע זיהוי הדיסקית וחישוב נקודת ה־XY. בנוסף, אתר המשחק פועל בסביבת Web רגילה ומשמש להזמנת משחק וקבלת קוד.

לכן בפרויקט קיימות שלוש סביבות עיקריות:
ESP32 לקוד המשובץ, Python לעיבוד תמונה, ו־אתר Web להזמנת משחק וניהול המשחק - במשתמש מנהל.

g. התייחסות לאבטחת מידע:

בפרויקט נשמרים נתוני משתמשים ומשחקים במערכת, ולכן נדרשת שמירה על פרטיות וגישה מבוקרת. משתמש יכול להתחיל משחק רק לאחר הכנסת **קוד משחק תקין**, שנבדק מול הנתונים השמורים במערכת.

סיסמאות המשתמשים אינן נשמרות כטקסט רגיל, אלא נשמרות לאחר ביצוע **Hash**. כלומר, במקום לשמור את הסיסמה עצמה, המערכת שומרת ערך מוצפן/מעובד חד־כיוונית, שלא אמור לאפשר שחזור של הסיסמה המקורית. בזמן התחברות, המערכת מבצעת Hash לסיסמה שהמשתמש הקליד ומשווה אותה לערך השמור.

16. פיתוחים עתידיים

בעתיד ניתן להרחיב את המערכת כך שתתמוך בכמה שולחנות משחק במקביל, ולא רק בשולחן אחד. כל שולחן יקבל מספר מזהה משלו, והמערכת תוכל לנהל עבורו קוד משחק, ניקוד, מצב משחק והיסטוריה נפרדת.

בנוסף, ניתן לשפר את דיוק זיהוי הדיסקית, להוסיף רמות משחק חכמות יותר, לשפר את מהירות תגובת הרובוט, ולהוסיף מערכת סטטיסטיקות לשחקנים באתר.
בעתיד אפשר גם להוסיף תמיכה בטורנירים או תחרות בין כמה שחקנים במספר שולחנות.

17. סיכום ומסקנות

בפרויקט נבנתה מערכת המדמה משחק הוקי אוויר מול רובוט. המערכת משלבת בין אתר להזמנת משחק, מסך מגע להפעלת המשחק, חיישנים, מצלמה, בקר ESP32, מנועי DC, מנוע סרוו, לדים ורמקול.

במהלך העבודה למדתי כיצד לחבר בין חומרה לתוכנה, כיצד לנהל משחק בזמן אמת, וכיצד להשתמש בנתונים מחיישנים וממצלמה כדי לגרום לרובוט להגיב למצב משתנה. בנוסף, למדתי להתמודד עם תכנון מכני, חיבור רכיבים, תקשורת עם Firebase וניהול מצבי משחק.

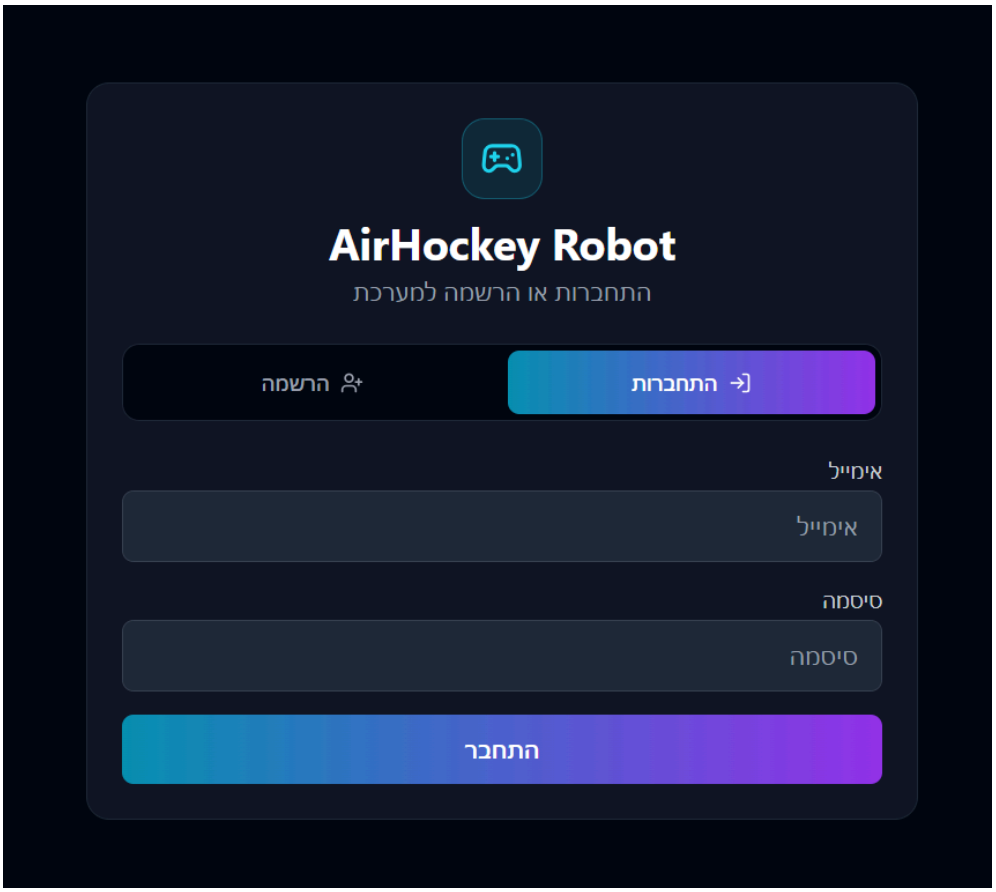
המסקנה העיקרית מהפרויקט היא שמערכת רובוטית דורשת שילוב מדויק בין כמה תחומים: מכניקה, אלקטרוניקה, תוכנה ובקרה. גם כאשר כל רכיב עובד בפני עצמו, האתגר המרכזי הוא לגרום לכל החלקים לעבוד יחד כמערכת אחת יציבה וברורה.

18. ביבליוגרפיה

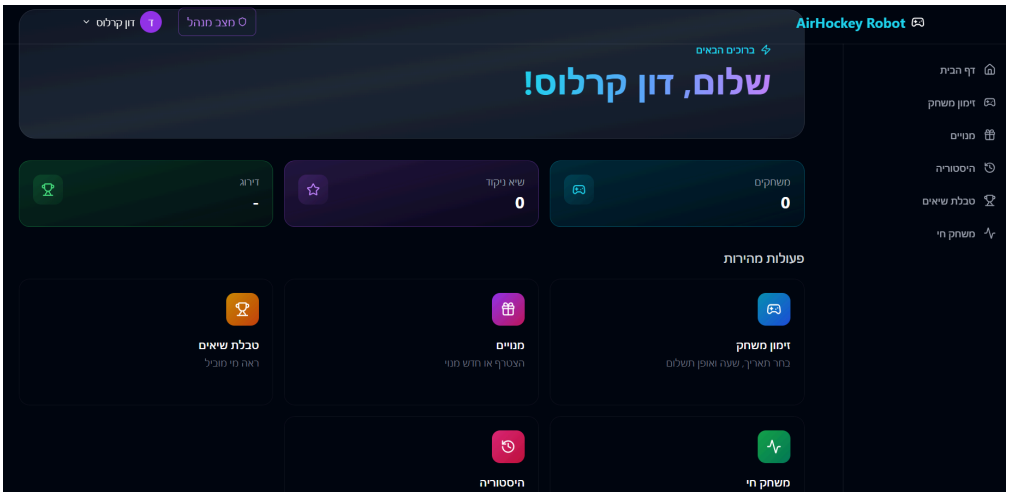
Espressif Systems — Arduino ESP32 Documentation
Firebase Documentation — Google
OpenCV Documentation — OpenCV-Python Tutorials
Arduino Documentation — WiFi Library

מסכי אפליקציה

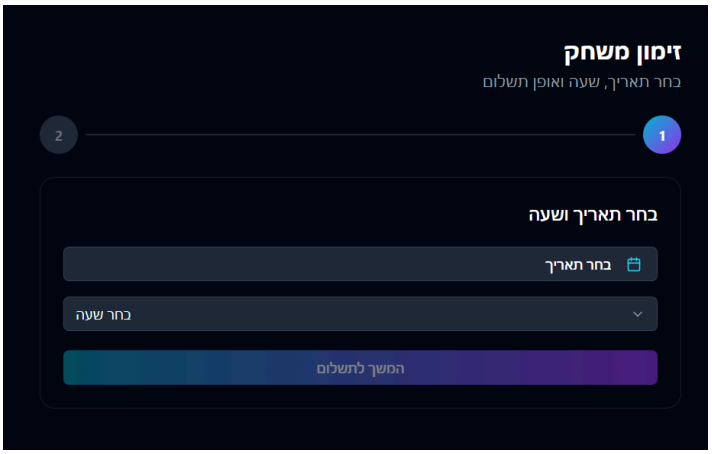
מסך כניסה:



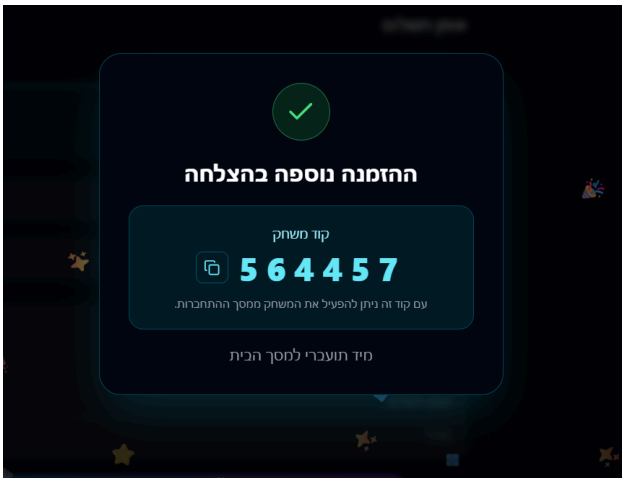
מסך בית



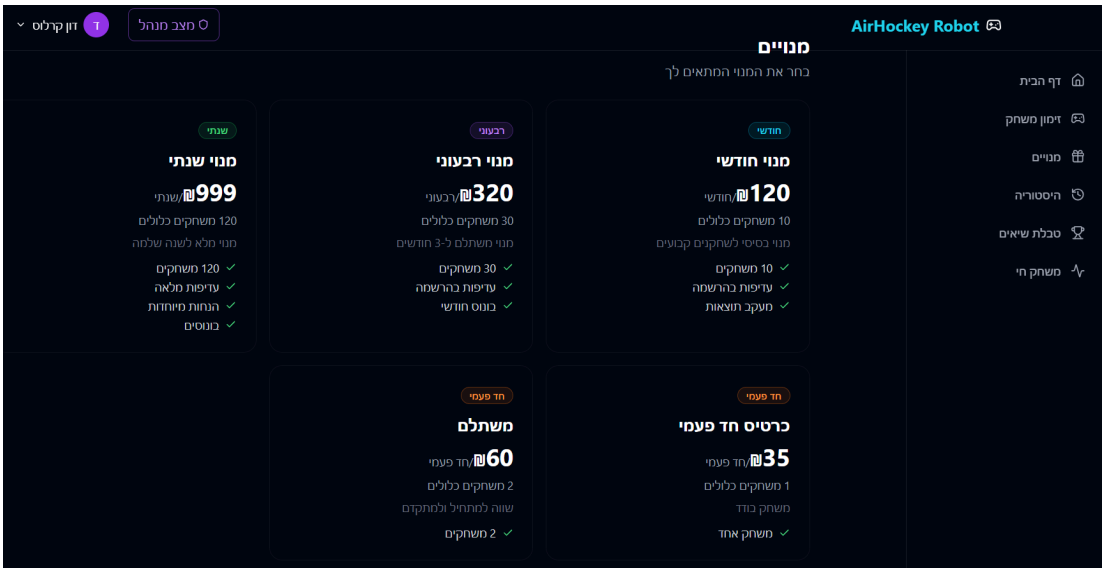
זימון משחק



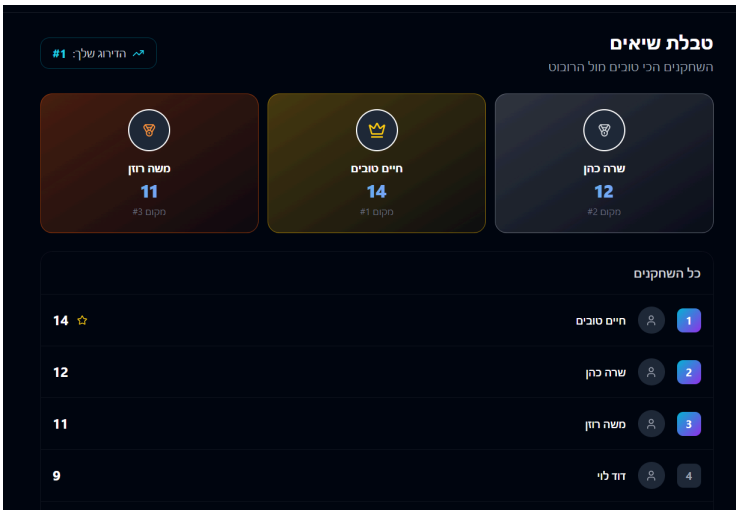
אישור הזמנת משחק



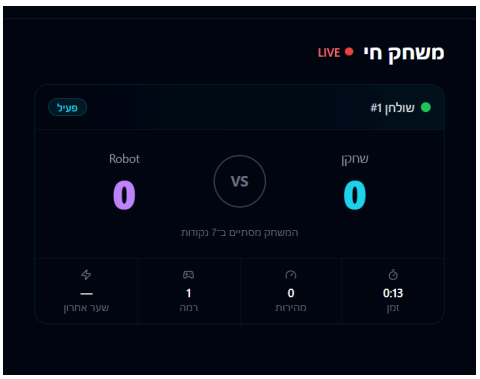
בחירת מנוי



טבלת שיאים



משחק חי

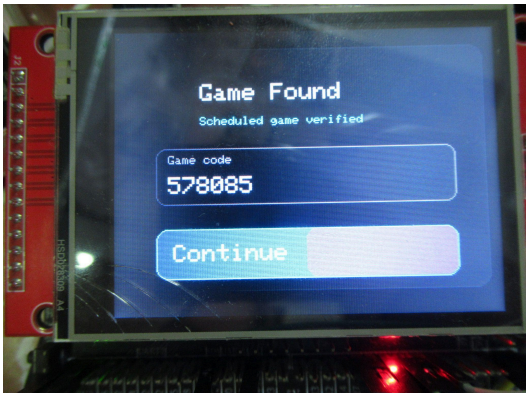


מסכי מסך מגע

1. הכנסת קוד משחק



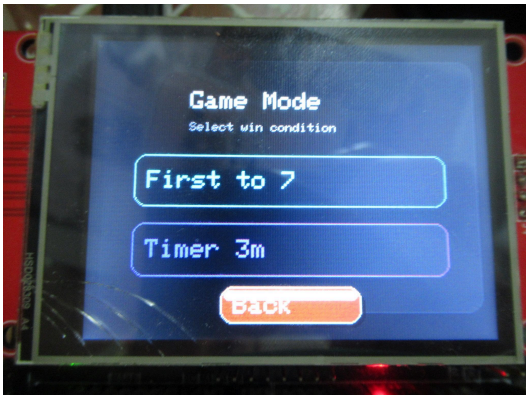
2. מציאת משחק



3. בחירת רמת משחק



4. בחירת סוג משחק



5. אישור הגדרות משחק



6. סיום משחק ומנצח

